Final Report

Style Transfer

Group members:

Name	ID
Danah Matuq Aldadi	445006203
Meran Awadh Aldaadi	445005774
Esraa Faisal Al Awam	445003185
Retal Fawaz Alhulayfi	445003015
Sadeem Nafaa Alsulami	445001695

Section Number: 2

Supervised by: Dr. Balsam Khojah

Table of Contents

Introduction

In recent years, deep learning has achieved remarkable success in computer vision, allowing machines to analyze, understand, and generate visual content. One important and creative application of deep learning is Neural Style Transfer, which transforms a normal image into an artistic image by combining the content of one image with the visual style of another. This technique has gained significant attention in digital art, image editing, and multimedia applications because it enables automatic artistic image generation without requiring advanced artistic skills or manual editing.

Neural Style Transfer provides an automated solution by using Convolutional Neural Networks (CNNs) to extract important visual features from images. Pretrained deep learning models, such as VGG19, can separate the structural representation of an image from its artistic style representation. In this project, the pretrained VGG19 model is used to extract content features from real-world landscape images and style features from selected artistic images. These extracted features are then combined to generate new artistic outputs that preserve the original image structure while applying artistic textures, colors, and patterns.

However, achieving a good balance between artistic style transfer and content preservation remains a major challenge in Neural Style Transfer systems. If the artistic style effect is too weak, the generated image may appear too similar to the original content image. On the other hand, if the style effect is too strong, important structural details and semantic information from the original image may be lost. In addition, some optimization settings may introduce visual noise, unstable convergence, or weak artistic representation, which can negatively affect the final output quality.

The goal of this project is to build a Neural Style Transfer system that converts landscape images into artistic images using deep learning techniques. The system starts with a baseline model and then applies several technical improvements to study how different configurations affect the generated outputs. These improvements include Total Variation Loss, Learning Rate Scheduler, Weighted Style Layers, optimizer tuning using Adam and LBFGS, Multi-Style blending, and hyperparameter adjustments. The effectiveness of the proposed system is evaluated using visual comparisons, loss curves, convergence analysis, and final loss values. Through these experiments, the project demonstrates how transfer learning and optimization techniques can improve artistic image generation while maintaining content structure and visual quality.

Methodology

1. Dataset

The dataset used in this project consists of two types of images: content images and style images. The content images were taken from the Landscape Pictures dataset on Kaggle [1], which contains real-world landscape photos. These images are used as the main structure that the model preserves during the style transfer process. The content dataset contains 4,319 landscape images collected from Kaggle. These images include mountains, rivers, lakes, forests, and other natural scenes. In addition, 9 style images were collected and used as style references for the style transfer process. For visualization and evaluation, 5 content images were randomly selected and shown in the report as sample examples. For the style images, a custom collection of artistic images was prepared and stored in a ZIP file. These images were selected manually based on their clear colors, strong textures, and visible artistic patterns, since these characteristics help produce more noticeable and visually effective style transfer results. Before applying the model, all images were preprocessed by resizing them to 256 × 256 pixels, converting them into tensors, and normalizing them using the standard VGG19 normalization values. A random subset of content images was selected from the dataset, and several style images were displayed to show examples of the data used in the project. The content images provide the original scene structure, while the style images provide the artistic colors, textures, and patterns that are transferred to the generated image.

Figure 1: Examples of content and style images used in the project

2. Preprocessing

To guarantee compatibility with the pre-trained VGG19 model, image preprocessing is a crucial step in this study. To ensure consistency, all input images, including content and style images, are resized to a uniform size of 256x256 pixels. Additionally, a separate experiment using 128×128 resolution was conducted to evaluate computational efficiency and convergence behavior. The images are then transformed into tensors using PyTorch transformations.

Additionally, the typical mean and standard deviation values of the ImageNet dataset are used for normalization (mean = [0.485, 0.456, 0.406], std = [0.229, 0.224, 0.225]). Because the VGG19 model was first trained on ImageNet, it is essential to match this distribution in order to enhance performance and the precision of feature extraction.

3. Model Architecture

A pre-trained convolutional neural network called the VGG19 model is utilized for feature extraction. The fully connected layers are removed from this project, leaving only the convolutional layers.

Simple features like edges are captured in shallow layers, while more complex features are captured in deeper layers. The model's parameters are fixed, making it behave like a set feature extractor for both style and content.

Images are processed through specific layers of VGG19 to extract features. The style features, such as textures and patterns, are captured by several layers (conv1_1 to conv5_1), whereas the content is represented by a deeper layer (conv4_2).

By measuring the relationships between feature maps, the Gram matrix is utilized to characterize image style. Rather than spatial details, it focuses on textures and patterns, which aids in the model's ability to convey artistic style.

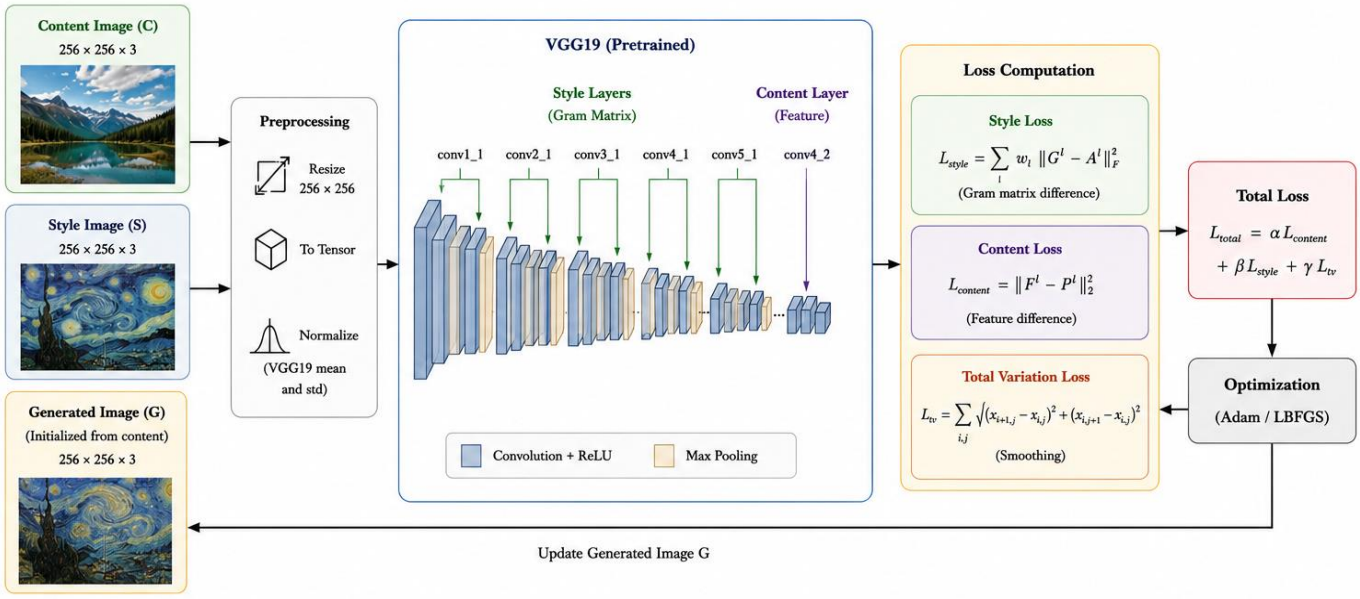

Figure 2: Neural Style Transfer pipeline using the pretrained VGG19 model for content and style feature extraction

4. Baseline Model

The baseline model represents the standard implementation of Neural Style Transfer without any additional improvements. The generated image is initialized as a copy of the content image and is then iteratively optimized to minimize the total loss function.

The overall loss is made up of two primary elements: style loss and content loss. The generated image retains the structure of the original content image because of content loss, and it captures the artistic style of the style image because of style loss.

The model is trained for 200 iterations using the Adam optimizer with a learning rate of 0.002. This basic configuration acts as a benchmark against which to compare the results of additional advancements and experiments.

5. Model Improvements

This section outlines the specific technical enhancements integrated into the baseline architecture to optimize the Neural Style Transfer process. These improvements focus on enhancing visual quality, ensuring training stability, and refining hyperparameter configurations to achieve superior artistic synthesis:

Improvement	Description (What Changed)	Purpose
Total Variation (TV) Loss	Added Total Variation Loss function to the optimization loop.	To reduce pixelated noise and spatial artifacts, resulting in a smoother output.
Learning rate Scheduler	Integrated a Learning Rate Scheduler (StepLR).	To improve training stability and ensure smooth convergence of the loss function.
Weighted Style	Applied unique weights across different VGG style layers.	To balance style details and ensure a more nuanced artistic representation.
Image Size 128	Utilized a 128 X 128 resolution for the experimental phase.	To speed up iterative testing and optimize computational memory usage.
LBFGS Optimizer	Replaced the Adam optimizer with LBFGS for image optimization.	To explore alternative optimization behavior and potentially achieve smoother convergence.
Changed Beta	Increased the style weight parameter (beta) from 5e5 to 8e5.	To enhance the influence of artistic style features in the generated image.
Changed Content Layer	Changed the content representation layer from conv4_2 to conv5_2.	To capture higher-level semantic features with less emphasis on fine spatial details.
Lower Learning Rate	Reduced the learning rate from 0.002 to 0.001.	To achieve more stable optimization and reduce sudden updates during training.
Multi-Style	Combined features from two different style images using blended style features from VGG19.	To generate artistic images that contain mixed visual features from multiple art styles.
Strong Style	Increased the style weight value beta during training.	To make the artistic style effect stronger and more visible in the generated image.

6. Results and Performance Analysis

In this section, we present a comprehensive evaluation of the model's performance. The analysis includes a visual comparison of the technical improvements, the convergence behavior of the loss functions, and the final stylized outputs across various content images.

6.1 Technical Improvements Comparison

The following figure illustrates the visual impact of each technical enhancement discussed in the methodology:

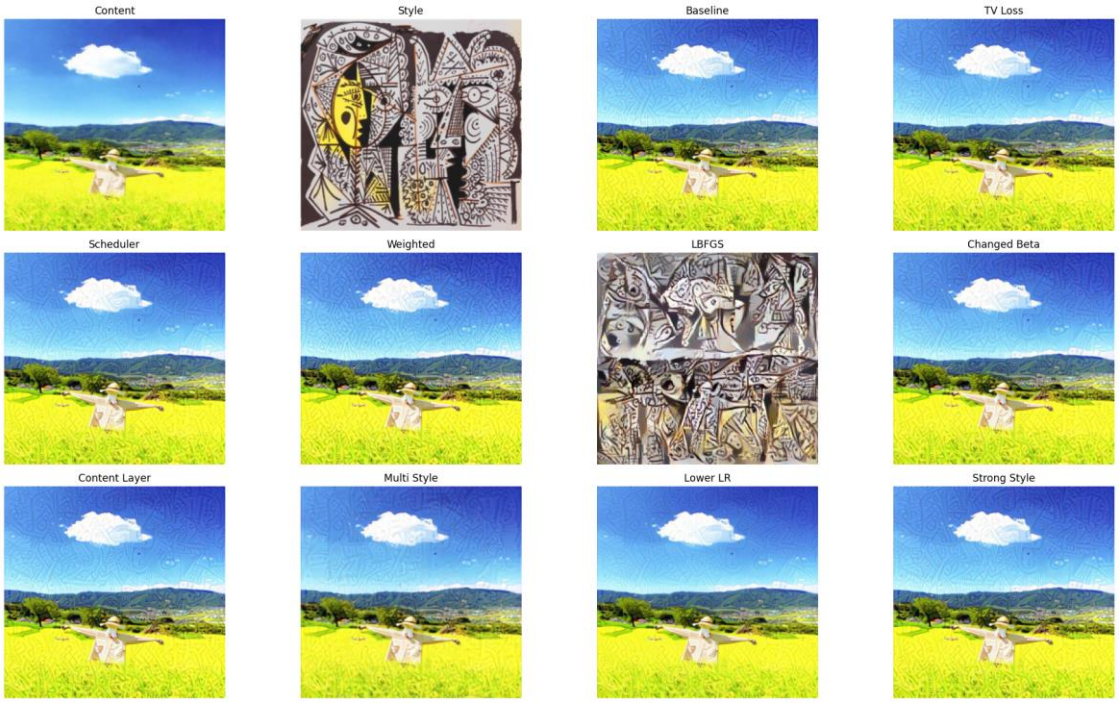

Figure 3: Visual comparison of different model configurations and improvements

Analysis:

- **TV Loss:** As observed, the addition of Total Variation loss significantly smoothed the image and removed the checkerboard artifacts present in the baseline.
- **LBFGS vs Adam:** The LBFGS optimizer produced a more textured and high-contrast style, while Adam maintained better structural stability for this specific configuration.
- **Multi-Style:** Combining multiple style features resulted in a unique blend of patterns while maintaining the content's semantic integrity.

- **Weighted Style:** Applying weighted style layers improved the balance between artistic texture and content preservation, producing visually pleasing results.
- **Lower Learning Rate:** Using a smaller learning rate resulted in more stable optimization but slower convergence.
- **Strong Style:** Increasing the style weight generated stronger artistic patterns, but reduced some content details.
- **Content Layer:** Modifying the content layer improved structural preservation of the original image.

6.2 Loss Curves and Convergence Analysis

To verify the stability of our training process, we monitored the total loss across 200 iterations for each experiment:

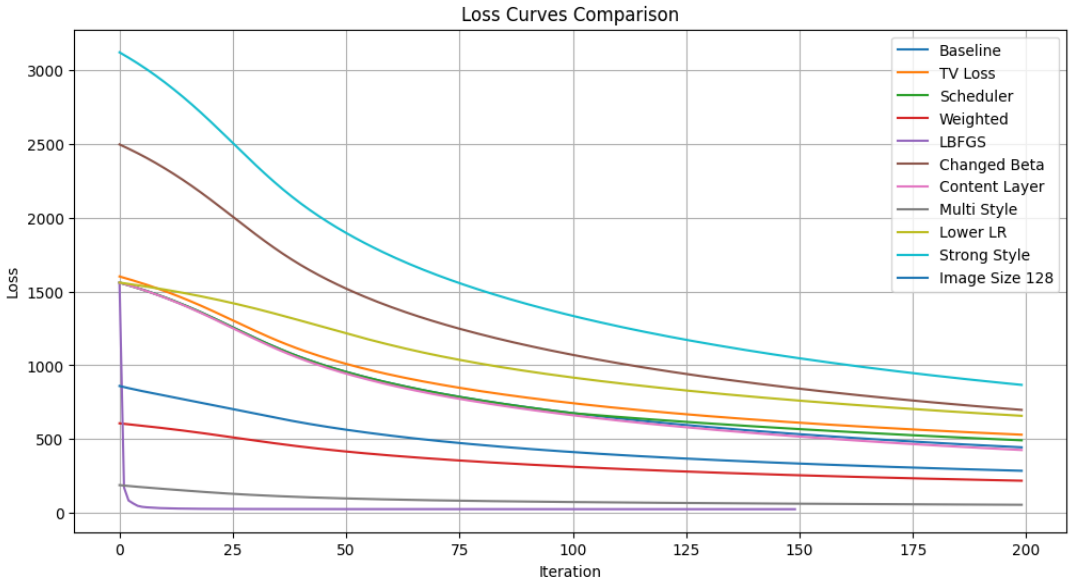

Figure 4: Convergence analysis of loss curves for all experimental configurations

Discussion:

The loss curves demonstrate that our Learning Rate Scheduler and Optimizer Tuning were successful. The LBFGS (purple line) shows the fastest convergence, reaching a stable state in under 25 iterations. Other configurations like Strong Style (cyan) and Weighted Style (red) show a steady and smooth decline, indicating a well-balanced optimization process without significant oscillations.

6.2.1 *Quantitative Performance Analysis*

This sub-section provides a detailed technical evaluation of the model's behavior during optimization:

- Optimization Speed: The LBFGS optimizer demonstrated exceptional efficiency, reducing the loss from 1,561.41 to 25.08 within the first 50 steps. This demonstrates its ability to converge faster than the Adam optimizer.
- Regularization & Smoothing: While the TV Loss started at a higher baseline (1,602.92), it provided a controlled descent. The final loss value reflects the trade-off between smoothing the image and minimizing the total loss.
- Structural Stability: Multi-Style configurations showed the highest stability among complex experiments, maintaining a low-loss trajectory throughout the process.

6.2.2 *Comparative Ranking of Configurations*

To evaluate the effectiveness of each enhancement, the configurations are ranked based on their Final Loss Value. This serves as a quantitative benchmark for convergence capability:

Rank	Configuration	Final Loss	Observation
1	LBFGS	24.48	Highest optimization precision
2	Multi-Style	54.53	Superior artistic blending
3	Weighted Style	217.69	Best visual balance and content preservation
4	Image Size 128	285.06	Faster convergence
5	Content Layer	425.94	Better content preservation
6	Baseline	444.16	Reference configuration
7	Scheduler	491.66	Smoother convergence
8	TV Loss	529.81	Reduced image noise
9	Lower LR	657.90	Stable but slower training
10	Changed Beta	698.15	Stronger style emphasis
11	Strong Style	867.68	Strong textures but weaker content details

6.3 Final Stylized Outputs

Although LBFGS achieved the lowest loss, the Weighted Style configuration with the Adam optimizer was selected for the final outputs because it produced better visual quality and preserved the content structure.

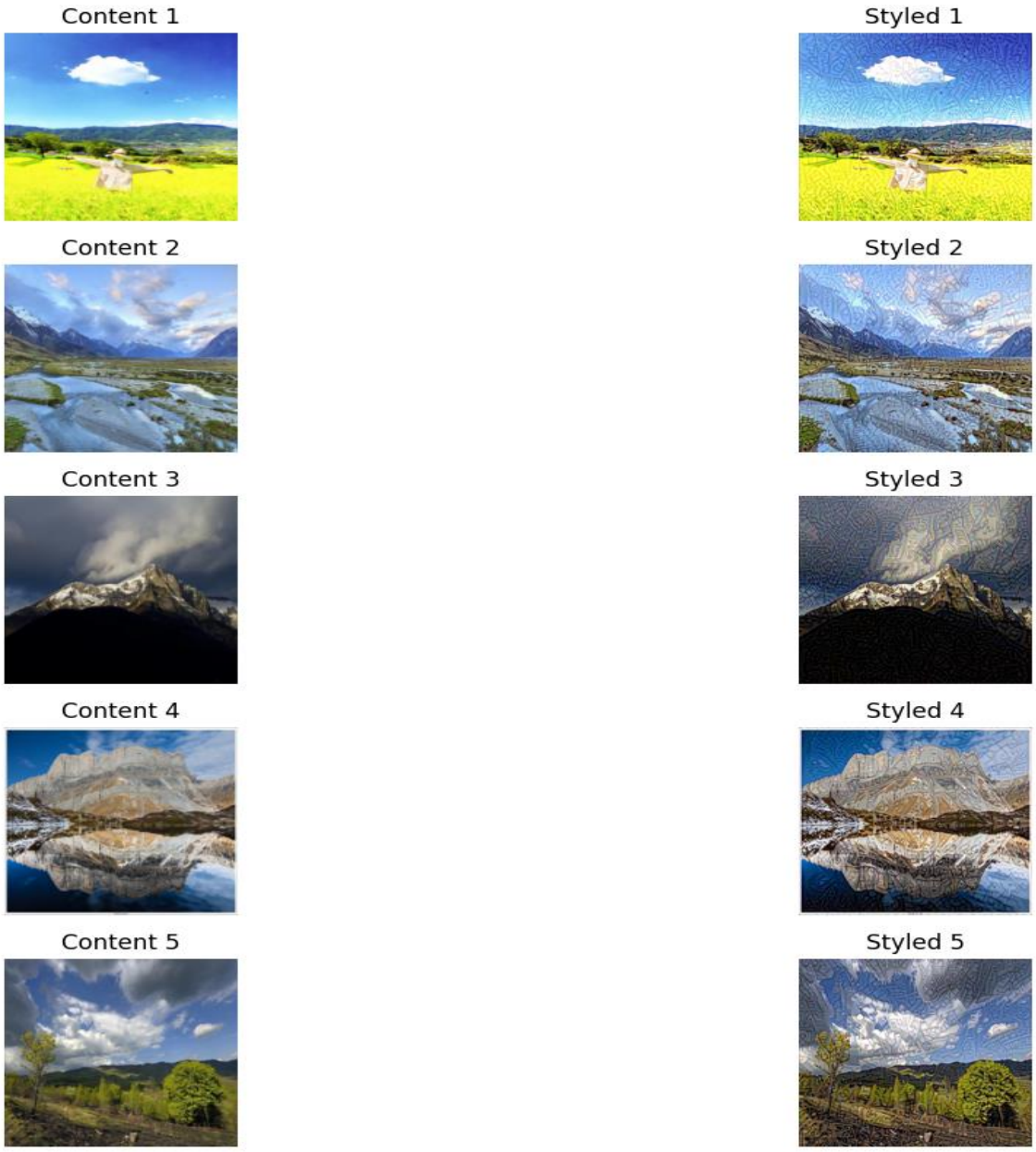

Figure 5: Final high-quality stylized results across diverse content types (landscapes and mountains)

Results Summary

The final outputs (Styled 1–5) demonstrate consistent quality. Across all test cases, the model successfully:

1. Transferred Artistic Features: Captured the intricate line work of the style image.
2. Preserved Global Structure: Kept the mountains and sky recognizable.
3. Efficiency: As shown in the logs, the loss for each image decreased significantly (e.g., Final Output 4 decreased from 506.87 to 58.65), proving the robustness of our final pipeline.

Limitations

- The quality of the generated output depends heavily on the selected style image.
- Lower image resolutions may reduce fine visual details in the final stylized image.
- The optimization process can be time-consuming, especially when using higher image resolutions or more iterations.
- The system is not suitable for real-time applications in its current implementation.
- Visual quality evaluation is subjective and may vary between users.

Conclusion

In this project, we implemented a Neural Style Transfer system using a pretrained VGG19 model to generate artistic images while preserving the main structure of the original content images. The system used landscape images as content inputs and selected artistic images as style references. By extracting content and style features from different VGG19 layers, the model was able to combine the structure of the content image with the colors, textures, and patterns of the style image.

Several improvements were applied and compared with the baseline model, including Total Variation Loss, Learning Rate Scheduler, Weighted Style Layers, Multi-Style blending, different style weights, changed content layers, and optimizer tuning using Adam and LBFGS. The results showed that these improvements affected the output quality in different ways. LBFGS achieved the fastest convergence and the lowest final loss, while Multi-Style blending produced more diverse artistic outputs. TV Loss helped reduce visual noise and artifacts, and Weighted Style Layers provided a better balance between content preservation and style representation.

Overall, the project demonstrated how deep learning and transfer learning can be used creatively for image generation tasks. The proposed system successfully generated stylized images across different landscape inputs and showed the importance of selecting suitable style images and optimization settings. Future work may include using larger and more diverse datasets, applying real-time style transfer, testing additional pretrained architectures, and exploring user-controlled style strength for more flexible artistic image generation.

References

[1] Arnaud58, "Landscape Pictures Dataset," Kaggle. [Online]. Available: https://www.kaggle.com/datasets/arnaud58/landscape-pictures. [Accessed: May 10, 2026].

[2] Adityajl, "Neural-Style-Transfer-Pytorch-," GitHub Repository. [Online]. Available: https://github.com/Adityajl/Neural-Style-Transfer-Pytorch- [Accessed: Aug 14, 2024].

[3] Nafis, N., "Neural Style Transfer using VGG19 and PyTorch," GitHub Repository. [Online]. Available: https://github.com/nazianafis/Neural-Style-Transfer [Accessed: July 6, 2023].

[4] L. A. Gatys, A. S. Ecker, and M. Bethge, "Image style transfer using convolutional neural networks," in Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016, pp. 2414–2423.

[5] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," arXiv preprint arXiv:1409.1556, 2014.

[6] V. Gupta, R. Sadana, and S. Moudgil, "Image style transfer using convolutional neural networks based on transfer learning," International Journal of Computational Systems Engineering, vol. 5, no. 1, pp. 53–60, 2019.

[7] X. Li, H. Cao, Z. Zhang, J. Hu, Y. Jin, and Z. Zhao, "Artistic neural style transfer algorithms with activation smoothing," in Proceedings of the 2025 2nd International Conference on Informatics Education and Computer Technology Applications, 2025, pp. 1–6.

[8] H. Liu, L. Wang, W. Guan, Y. Zhang, and Y. Guo, "Pluggable style representation learning for multi-style transfer," in Proceedings of the Asian Conference on Computer Vision, 2024, pp. 2087–2104.

[9] M. Reyad, A. M. Sarhan, and M. Arafa, "A modified Adam algorithm for deep neural network optimization," Neural Computing and Applications, vol. 35, no. 23, pp. 17095–17112, 2023.

[10] K. Kashyap, M. Garg, S. Fargose, and S. Nair, "Dynamic neural style transfer for artistic image generation using VGG19," arXiv preprint arXiv:2501.09420, 2025.